Compte rendu de TP

Currency Converter — Application Android en Kotlin

1. Introduction et objectifs

Ce travail pratique a consisté à développer une application Android native en **Kotlin** intitulée « **Currency Converter** ». Elle convertit un montant d'une devise source vers une devise cible à partir de taux de change fixes, en passant par l'euro comme devise pivot.

Les principaux objectifs atteints sont :

- la construction d'une interface adaptative avec `ConstraintLayout` et `LinearLayout` ;
- l'utilisation de composants interactifs (`Spinner`, `ArrayAdapter`, `ImageButton`) ;
- la mise en place d'un clavier numérique personnalisé intégré à l'application ;
- l'application rigoureuse de règles de gestion (validation, formatage, limitation de saisie) ;
- l'utilisation du **View Binding** pour un accès sécurisé aux vues.

2. Choix d'interface (activity_main.xml)

L'interface respecte la maquette fournie et se divise en deux zones.

Zone supérieure (environ 52 % de l'écran)

- Fond bleu turquoise uni (#2BB8E8).
- Elle contient le titre, les deux sélecteurs de devises (`spDeviseSource` et `spDeviseCible`), l'affichage du montant saisi et du résultat (police 44 sp, alignés à droite et soulignés), le bouton d'inversion et le bouton *Convertir*.
- Le bouton d'inversion repose sur un fond circulaire blanc (`bg_circle.xml`) pour rester bien visible sur le fond turquoise.

Zone inférieure : clavier numérique

- Un `LinearLayout` organisé en 4 lignes × 3 colonnes simule le pavé numérique.
- Les chiffres sont en bleu turquoise (#1BA6C9) ; la touche de remise à zéro « **C** » est en rouge clair (#F0727A).
- Les boutons sont sans bordure (`borderlessButtonStyle`) en 32 sp.
- Une `Guideline` à 52 % sépare nettement la zone d'affichage du clavier.

3. Logique de conversion (MainActivity.kt)

A. Listes et taux fixes

Les deux `Spinner` sont alimentés par un `ArrayAdapter` construit sur la ressource `R.array.devises` (USD, EUR, TND, GBP, JPY, CAD). Par défaut, la source est USD et la cible EUR. Les taux pour 1 EUR sont stockés dans une `Map<String, Double>` :

```
private val taux = mapOf(
    "EUR" to 1.0, "USD" to 1.17, "TND" to 3.40,
    "GBP" to 0.87, "JPY" to 172.50, "CAD" to 1.62
)
```

B. Algorithme de conversion (devise pivot)

La fonction de conversion procède en trois étapes :

1. Montant en EUR = montant ÷ taux(source)
2. Résultat = montant en EUR × taux(cible)
3. Le résultat est arrondi à 2 décimales avec `String.format(Locale.getDefault(), "%.2f", resultat)`.

Exemple : 120 USD → EUR donne 120 ÷ 1,17 = 102,5641…, soit **102.56** après arrondi.

C. Clavier et règles de gestion

- **Saisie personnalisée :** `tvMontant` est un `TextView`, donc aucun clavier système n'apparaît. Chaque touche appelle `ajouterCaractere(c)`.
- **Zéro initial :** le montant de départ est « 0 » ; le premier chiffre tapé le remplace.
- **Point décimal unique :** une seconde pression sur « . » est ignorée.
- **Longueur limitée** à 10 caractères.
- **Validation :** si le montant vaut 0 au clic sur *Convertir*, un `Toast` « Veuillez saisir un montant » s'affiche.
- **Touche « C » :** remet le montant à « 0 » et vide le résultat.
- **Inversion :** `btnInverser` échange les sélections des deux `Spinner` puis relance la conversion.
- **Bonus :** conversion automatique à chaque appui sur le clavier ou changement de devise.

4. Cas de test et résultats

Montant	Source	Cible	Résultat attendu	Résultat obtenu	Statut
120	USD	EUR	102.56	102.56	✓ Validé
100	EUR	TND	340.00	340.00	✓ Validé
50	GBP	USD	67.24	67.24	✓ Validé
10	EUR	EUR	10.00	10.00	✓ Validé

Captures d'écran des cas de test



Test 1 : 120 USD → EUR *Test 2 : 100 EUR → TND* *Test 3 : 50 GBP → USD* *Test 4 : 10 EUR → EUR*

5. Difficultés rencontrées et solutions

1. Agencement dans le ConstraintLayout. Placer côte à côte les listes et les montants avec un bouton d'inversion vertical risquait de provoquer des chevauchements. *Solution :* une structure hiérarchique claire, avec une `Guideline` à 52 % séparant la zone d'affichage et le clavier.

2. Visibilité des icônes vectorielles. L'icône d'inversion se distinguait mal sur le fond turquoise. *Solution :* ajout d'un fond circulaire blanc (`bg_circle.xml`) sous le bouton pour assurer un contraste optimal.

6. Conclusion

Ce TP a permis de réaliser une application Android interactive complète en Kotlin. Le respect de la maquette, la rigueur dans la gestion des règles de saisie et une logique de calcul fiable (devise pivot) offrent une expérience fluide. Les quatre cas de test demandés donnent les résultats attendus.